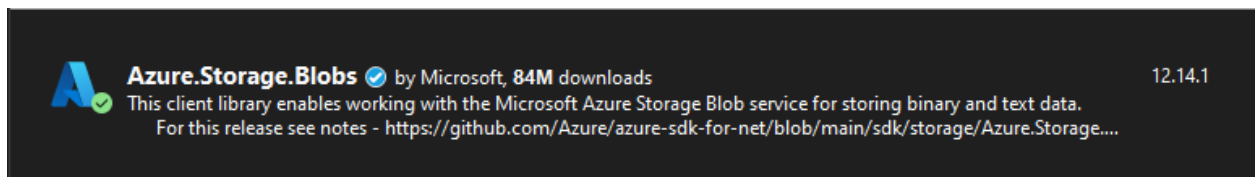


1. Create the Web API Project and create the structure.
2. Create the interface in Services folder and define the contracts

```
public interface IBlobService
{
    Task<BlobDetails> GetBlobAsync(string blobName);
    Task DeleteBlobAsync(string blobName);
    Task<IEnumerable<string>> ListAllBlobsAsync();
    Task UploadContentToBlob(string content, string fileName);
    Task UploadFileToBlob(string filePath, string fileName);
}
```

3. Install nuget packages → AzureStorage.Blobs



4. Create storage account and blob storage (But don't allow public access)

+ Container 🔒 Change access level ↶ Restore co

Search containers by prefix

Name
☐ \$logs
☐ dotnet031

New container ✕

Name *

dotnet031 ✓

Public access level ⓘ

Container (anonymous read access for containers and blobs) ▼

 All container and blob data can be read by anonymous request. Clients can enumerate blobs within the container by anonymous request, but cannot enumerate containers within the storage account.

▼ Advanced

5. Show file upload through portal.

⬆ Upload 🔒 Change access level 🔄 Refresh | 🗑 Delete ↔ Change tier 🔑 Acquire lease

Authentication method: Access key ([Switch to Azure AD User Account](#))
Location: dotnet031

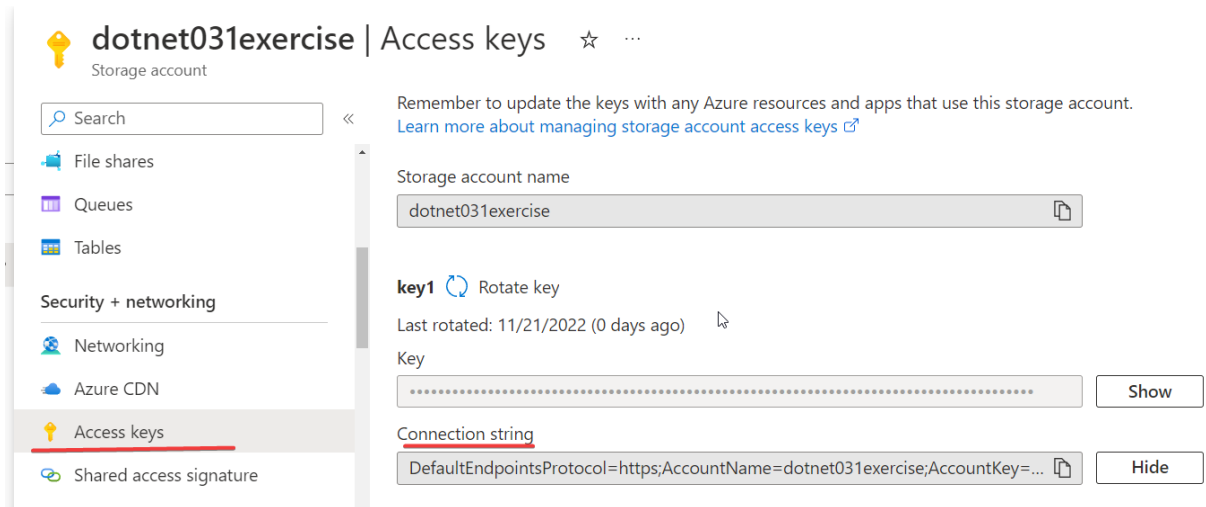
Search blobs by prefix (case-sensitive)

+🔍 Add filter

Name	Modified	Access tier
☐  Azure logo.png	11/21/2022, 1:57:53 ...	Hot (Inferred)

6. Go to the storage account level, click on access keys and copy the connection string

<Your Key Goes Here>



7. Add the connection string in Appsettings.json

```
{
  "AzureBlobStorageConString": "DefaultEndpointsProtocol=https;AccountName=dotnet031exercise;AccountKey=...;EndpointSuffix=core.windows.net",
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

8. Use the to register the service in the Program.cs. This includes adding the blob service client and registering the IBlobService. Note: BlobServiceClient should be injected as a dependency.

```
//Configure the BlobServiceClient
builder.Services.AddSingleton(x:IServiceProvider =>
    new BlobServiceClient(connectionString: builder.Configuration // ConfigurationManager
        .GetValue<string>(key: "AzureBlobStorageConString")));

//Inject the dependencies
builder.Services.AddSingleton<IBlobService, BlobService>();
```

9. In the constructor of BlobService inject the dependency. (If you need to keep the flexibility of choosing the blob name, don't set the _containerClient in the constructor.

```

public class BlobService:IBlobService
{
    private readonly BlobServiceClient _blobServiceClient;
    private readonly BlobContainerClient _containerClient;

    0 references
    public BlobService(BlobServiceClient blobServiceClient)
    {
        _blobServiceClient = blobServiceClient;
        _containerClient = _blobServiceClient.GetBlobContainerClient("containerName");
    }
}

```

10. Creating the additional models.

BlobDetails

```

public class BlobDetails
{
    public Stream? Content { get; set; }
    public string ContentType { get; set; } = string.Empty;

    public BlobDetails(Stream? content, string contentType)
    {
        Content=content;
        ContentType=contentType;
    }
}

```

UploadContentRequest

```

public class UploadContentRequest
{
    public string FileName { get; set; }
    public string Content { get; set; }
}

```

UploadFileRequest

```

public class UploadFileRequest
{
    public string FilePath { get; set; } // same path of the file in the drive
    public string FileName { get; set; } //The name which the file should be named
    in Blob Storage
}

```

11. Creating the Extension Method.

```
public static class FileExtensions
{
    private static readonly FileExtensionContentTypeProvider _provider = new
FileExtensionContentTypeProvider();

    public static string GetContentType(this string fileName)
    {
        if (!_provider.TryGetContentType(fileName, out var contentType))
        {
            contentType = "application/octet-stream";
        }

        return contentType;
    }
}
```

12. Implementing IBlobService

```

public class BlobService:IBlobService
{
    private readonly BlobServiceClient _blobServiceClient;
    private readonly BlobContainerClient _containerClient;

    public BlobService(BlobServiceClient blobServiceClient)
    {
        _blobServiceClient = blobServiceClient;
        _containerClient = _blobServiceClient.GetBlobContainerClient("my-image-con-
tainer");
    }

    public async Task<BlobDetails> GetBlobAsync(string blobName)
    {
        var blobClient = _containerClient.GetBlobClient(blobName);
        var blobDownloadInfo = await blobClient.DownloadAsync();

        return new BlobDetails(blobDownloadInfo.Value.Content, blobDown-
loadInfo.Value.ContentType);
    }

    public async Task DeleteBlobAsync(string blobName)
    {
        var blobClient = _containerClient.GetBlobClient(blobName);
        await blobClient.DeleteIfExistsAsync();
    }

    public async Task<IEnumerable<string>> ListAllBlobsAsync()
    {
        var items = new List<string>();

        await foreach (var blobItem in _containerClient.GetBlobsAsync())
        {
            items.Add(blobItem.Name);
        }
        return items;
    }

    public async Task UploadContentToBlob(string content, string fileName)
    {
        var blobClient= _containerClient.GetBlobClient(fileName);
        var bytes=Encoding.UTF8.GetBytes(content);

        await using var memoryStream = new MemoryStream(bytes);
        await blobClient.UploadAsync(memoryStream, new BlobHttpHeaders { ContentType
= fileName.GetContentType() });
    }

    public async Task UploadFileToBlob(string filePath, string fileName)
    {
        var blobClient = _containerClient.GetBlobClient(fileName);
        await blobClient.UploadAsync(Path.Combine(filePath, fileName), new BlobHttp-
Headers { ContentType = Path.Combine(filePath, fileName).GetContentType() });
    }
}

```

13. BlobDemoController Code

```

namespace AzureStorage_BlobAPI.Controllers
{
    [Route("blobs")]
    [ApiController]
    public class BlobDemoController : ControllerBase
    {
        private readonly IBlobService _blobService;
        public BlobDemoController(IBlobService blobService)
        {
            _blobService = blobService;
        }

        [HttpGet("{blobName}")]
        public async Task<IActionResult> GetBlob(string blobName)
        {
            var data = await _blobService.GetBlobAsync(blobName);
            return File(data.Content, data.ContentType);
        }

        [HttpGet("list")]
        public async Task<IActionResult> ListBlobs()
        {
            return Ok(await _blobService.ListAllBlobsAsync());
        }

        [HttpPost("uploadfile")]
        public async Task<IActionResult> UploadFile([FromBody] UploadFileRequest re-
quest)
        {
            await _blobService.UploadFileToBlob(request.FilePath, request.FileName);
            return Ok();
        }

        [HttpPost("uploadcontent")]
        public async Task<IActionResult> UploadContent([FromBody] UploadContentRe-
quest request)
        {
            await _blobService.UploadContentToBlob(request.Content, request.File-
Name);
            return Ok();
        }

        [Route("{blobName}")]
        [HttpDelete]
        public async Task<IActionResult> DeleteFile(string blobName)
        {
            await _blobService.DeleteBlobAsync(blobName);
            return Ok();
        }
    }
}

```